

Appendix B. Common x86 Instructions

Reading the Tables

The tables of instructions presented in this appendix include:

- The instruction code
- The operands used
- The flags used
- A brief description of what the instruction does

In the operands section, it will list the type of operands it takes. If it takes more than one operand, each operand will be separated by a comma. Each operand will have a list of codes which tell whether the operand can be an immediate-mode value (I), a register (R), or a memory address (M). For example, the `movl` instruction is listed as `I/R/M, R/M`. This means that the first operand can be any kind of value, while the second operand must be a register or memory location. Note, however, that in x86 assembly language you cannot have more than one operand be a memory location.

In the flags section, it lists the flags in the `%eflags` register affected by the instruction. The following flags are mentioned:

O

Overflow flag. This is set to true if the destination operand was not large enough to hold the result of the instruction.

S

Sign flag. This is set to the sign of the last result.

Appendix B. Common x86 Instructions

Z

Zero flag. This flag is set to true if the result of the instruction is zero.

A

Auxiliary carry flag. This flag is set for carries and borrows between the third and fourth bit. It is not often used.

P

Parity flag. This flag is set to true if the low byte of the last result had an even number of 1 bits.

C

Carry flag. Used in arithmetic to say whether or not the result should be carried over to an additional byte. If the carry flag is set, that usually means that the destination register could not hold the full result. It is up to the programmer to decide on what action to take (i.e. - propagate the result to another byte, signal an error, or ignore it entirely).

Other flags exist, but they are much less important.

Data Transfer Instructions

These instructions perform little, if any computation. Instead they are mostly used for moving data from one place to another.

Table B-1. Data Transfer Instructions

Instruction	Operands	Affected Flags
movl	I/R/M, I/R/M	O/S/Z/A/C
This copies a word of data from one location to another. <code>movl %eax, %ebx</code> copies the contents of %eax to %ebx		

Instruction	Operands	Affected Flags
<code>movb</code>	I/R/M, I/R/M	O/S/Z/A/C
Same as <code>movl</code> , but operates on individual bytes.		
<code>leal</code>	M, I/R/M	O/S/Z/A/C
This takes a memory location given in the standard format, and, instead of loading the contents of the memory location, loads the computed address. For example, <code>leal 5(%ebp,%ecx,1), %eax</code> loads the address computed by $5 + \%ebp + 1 * \%ecx$ and stores that in <code>%eax</code>		
<code>popl</code>	R/M	O/S/Z/A/C
Pops the top of the stack into the given location. This is equivalent to performing <code>movl (%esp), R/M</code> followed by <code>addl \$4, %esp</code> . <code>popfl</code> is a variant which pops the top of the stack into the <code>%eflags</code> register.		
<code>pushl</code>	I/R/M	O/S/Z/A/C
Pushes the given value onto the stack. This is the equivalent to performing <code>subl \$4, %esp</code> followed by <code>movl I/R/M, (%esp)</code> . <code>pushfl</code> is a variant which pushes the current contents of the <code>%eflags</code> register onto the top of the stack.		
<code>xchgl</code>	R/M, R/M	O/S/Z/A/C
Exchange the values of the given operands.		

Integer Instructions

These are basic calculating instructions that operate on signed or unsigned integers.

Table B-2. Integer Instructions

Instruction	Operands	Affected Flags
<code>adcl</code>	I/R/M, R/M	O/S/Z/A/P/C

Appendix B. Common x86 Instructions

Instruction	Operands	Affected Flags
Add with carry. Adds the carry bit and the first operand to the second, and, if there is an overflow, sets overflow and carry to true. This is usually used for operations larger than a machine word. The addition on the least-significant word would take place using <code>addl</code> , while additions to the other words would use the <code>adc</code> instruction to take the carry from the previous <code>add</code> into account. For the usual case, this is not used, and <code>addl</code> is used instead.		
<code>addl</code>	I/R/M, R/M	O/S/Z/A/P/C
Addition. Adds the first operand to the second, storing the result in the second. If the result is larger than the destination register, the overflow and carry bits are set to true. This instruction operates on both signed and unsigned integers.		
<code>cdq</code>		O/S/Z/A/P/C
Converts the <code>%eax</code> word into the double-word consisting of <code>%edx:%eax</code> with sign extension. The <code>q</code> signifies that it is a <i>quad-word</i> . It's actually a double-word, but it's called a quad-word because of the terminology used in the 16-bit days. This is usually used before issuing an <code>idivl</code> instruction.		
<code>cmpl</code>	I/R/M, R/M	O/S/Z/A/P/C
Compares two integers. It does this by subtracting the first operand from the second. It discards the results, but sets the flags accordingly. Usually used before a conditional jump.		
<code>decl</code>	R/M	O/S/Z/A/P
Decrements the register or memory location. Use <code>decb</code> to decrement a byte instead of a word.		
<code>divl</code>	R/M	O/S/Z/A/P
Performs unsigned division. Divides the contents of the double-word contained in the combined <code>%edx:%eax</code> registers by the value in the register or memory location specified. The <code>%eax</code> register contains the resulting quotient, and the <code>%edx</code> register contains the resulting remainder. If the quotient is too large to fit in <code>%eax</code> , it triggers a type 0 interrupt.		
<code>idivl</code>	R/M	O/S/Z/A/P

Instruction	Operands	Affected Flags
Performs signed division. Operates just like <code>divl</code> above.		
<code>imull</code>	R/M/I, R	O/S/Z/A/P/C
Performs signed multiplication and stores the result in the second operand. If the second operand is left out, it is assumed to be <code>%eax</code> , and the full result is stored in the double-word <code>%edx:%eax</code> .		
<code>incl</code>	R/M	O/S/Z/A/P
Increments the given register or memory location. Use <code>incb</code> to increment a byte instead of a word.		
<code>mull</code>	R/M/I, R	O/S/Z/A/P/C
Perform unsigned multiplication. Same rules as apply to <code>imull</code> .		
<code>negl</code>	R/M	O/S/Z/A/P/C
Negates (gives the two's complement inversion of) the given register or memory location.		
<code>sbb</code>	I/R/M, R/M	O/S/Z/A/P/C
Subtract with borrowing. This is used in the same way that <code>adc</code> is, except for subtraction. Normally only <code>subl</code> is used.		
<code>subl</code>	I/R/M, R/M	O/S/Z/A/P/C
Subtract the two operands. This subtracts the first operand from the second, and stores the result in the second operand. This instruction can be used on both signed and unsigned numbers.		

Logic Instructions

These instructions operate on memory as bits instead of words.

Table B-3. Logic Instructions

Instruction	Operands	Affected Flags
-------------	----------	----------------

Appendix B. Common x86 Instructions

Instruction	Operands	Affected Flags
andl	I/R/M, R/M	O/S/Z/P/C
Performs a logical and of the contents of the two operands, and stores the result in the second operand. Sets the overflow and carry flags to false.		
notl	R/M	
Performs a logical not on each bit in the operand. Also known as a one's complement.		
orl	I/R/M, R/M	O/S/Z/A/P/C
Performs a logical or between the two operands, and stores the result in the second operand. Sets the overflow and carry flags to false.		
rcll	I/%c l, R/M	O/C
Rotates the given location's bits to the left the number of times in the first operand, which is either an immediate-mode value or the register %c l. The carry flag is included in the rotation, making it use 33 bits instead of 32. Also sets the overflow flag.		
rcrl	I/%c l, R/M	O/C
Same as above, but rotates right.		
roll	I/%c l, R/M	O/C
Rotate bits to the left. It sets the overflow and carry flags, but does not count the carry flag as part of the rotation. The number of bits to roll is either specified in immediate mode or is contained in the %c l register.		
rorl	I/%c l, R/M	O/C
Same as above, but rotates right.		
sall	I/%c l, R/M	C
Arithmetic shift left. The sign bit is shifted out to the carry flag, and a zero bit is placed in the least significant bit. Other bits are simply shifted to the left. This is the same as the regular shift left. The number of bits to shift is either specified in immediate mode or is contained in the %c l register.		
sarl	I/%c l, R/M	C

Instruction	Operands	Affected Flags
Arithmetic shift right. The least significant bit is shifted out to the carry flag. The sign bit is shifted in, and kept as the sign bit. Other bits are simply shifted to the right. The number of bits to shift is either specified in immediate mode or is contained in the %c l register.		
shll	I/%c l, R/M	C
Logical shift left. This shifts all bits to the left (sign bit is not treated specially). The leftmost bit is pushed to the carry flag. The number of bits to shift is either specified in immediate mode or is contained in the %c l register.		
shrl	I/%c l, R/M	C
Logical shift right. This shifts all bits in the register to the right (sign bit is not treated specially). The rightmost bit is pushed to the carry flag. The number of bits to shift is either specified in immediate mode or is contained in the %c l register.		
testl	I/R/M, R/M	O/S/Z/A/P/C
Does a logical and of both operands and discards the results, but sets the flags accordingly.		
xorl	I/R/M, R/M	O/S/Z/A/P/C
Does an exclusive or on the two operands, and stores the result in the second operand. Sets the overflow and carry flags to false.		

Flow Control Instructions

These instructions may alter the flow of the program.

Table B-4. Flow Control Instructions

Instruction	Operands	Affected Flags
call	destination address	O/S/Z/A/C

Appendix B. Common x86 Instructions

Instruction	Operands	Affected Flags
This pushes what would be the next value for %eip onto the stack, and jumps to the destination address. Used for function calls. Alternatively, the destination address can be an asterisk followed by a register for an indirect function call. For example, <code>call *%eax</code> will call the function at the address in %eax.		
int	I	O/S/Z/A/C
Causes an interrupt of the given number. This is usually used for system calls and other kernel interfaces.		
Jcc	destination address	O/S/Z/A/C

Instruction	Operands	Affected Flags
Conditional branch. <i>cc</i> is the <i>condition code</i>. Jumps to the given address if the condition code is true (set from the previous instruction, probably a comparison). Otherwise, goes to the next instruction. The condition codes are: • [n]a[e] - above (unsigned greater than). An <i>n</i> can be added for "not" and an <i>e</i> can be added for "or equal to" • [n]b[e] - below (unsigned less than) • [n]e - equal to • [n]z - zero • [n]g[e] - greater than (signed comparison) • [n]l[e] - less than (signed comparison) • [n]c - carry flag set • [n]o - overflow flag set • [p]p - parity flag set • [n]s - sign flag set • ecxz - %ecx is zero		
jmp	destination address	O/S/Z/A/C
An unconditional jump. This simply sets %<i>ip</i> to the destination address. Alternatively, the destination address can be an asterisk followed by a register for an indirect jump. For example, <code>jmp *%eax</code> will jump to the address in %<i>eax</i>.		

Appendix B. Common x86 Instructions

Instruction	Operands	Affected Flags
ret		O/S/Z/A/C
Pops a value off of the stack and then sets %eip to that value. Used to return from function calls.		

Assembler Directives

These are instructions to the assembler and linker, instead of instructions to the processor. These are used to help the assembler put your code together properly, and make it easier to use.

Table B-5. Assembler Directives

Directive	Operands	
.ascii	QUOTED STRING	
Takes the given quoted string and converts it into byte data.		
.byte	VALUES	
Takes a comma-separated list of values and inserts them right there in the program as data.		
.endr		
Ends a repeating section defined with .rept.		
.equ	LABEL, VALUE	
Sets the given label equivalent to the given value. The value can be a number, a character, or an constant expression that evaluates to a a number or character. From that point on, use of the label will be substituted for the given value.		
.globl	LABEL	
Sets the given label as global, meaning that it can be used from separately-compiled object files.		
.include	FILE	

Directive	Operands	
Includes the given file just as if it were typed in right there.		
<code>.lcomm</code>	<code>SYMBOL, SIZE</code>	
This is used in the <code>.bss</code> section to specify storage that should be allocated when the program is executed. Defines the symbol with the address where the storage will be located, and makes sure that it is the given number of bytes long.		
<code>.long</code>	<code>VALUES</code>	
Takes a sequence of numbers separated by commas, and inserts those numbers as 4-byte words right where they are in the program.		
<code>.rept</code>	<code>COUNT</code>	
Repeats everything between this directive and the <code>.endr</code> directives the number of times specified.		
<code>.section</code>	<code>SECTION NAME</code>	
Switches the section that is being worked on. Common sections include <code>.text</code> (for code), <code>.data</code> (for data embedded in the program itself), and <code>.bss</code> (for uninitialized global data).		
<code>.type</code>	<code>SYMBOL, @function</code>	
Tells the linker that the given symbol is a function.		

Differences in Other Syntaxes and Terminology

The syntax for assembly language used in this book is known as the *AT&T* syntax. It is the one supported by the GNU tool chain that comes standard with every Linux distribution. However, the official syntax for x86 assembly language (known as the Intel® syntax) is different. It is the same assembly language for the same platform, but it looks different. Some of the differences include:

- In Intel syntax, the operands of instructions are often reversed. The destination operand is listed before the source operand.

Appendix B. Common x86 Instructions

- In Intel syntax, registers are not prefixed with the percent sign (%).
- In Intel syntax, a dollar-sign (\$) is not required to do immediate-mode addressing. Instead, non-immediate addressing is accomplished by surrounding the address with brackets ([]).
- In Intel syntax, the instruction name does not include the size of data being moved. If that is ambiguous, it is explicitly stated as BYTE, WORD, or DWORD immediately after the instruction name.
- The way that memory addresses are represented in Intel assembly language is much different (shown below).
- Because the x86 processor line originally started out as a 16-bit processor, most literature about x86 processors refer to words as 16-bit values, and call 32-bit values double words. However, we use the term "word" to refer to the standard register size on a processor, which is 32 bits on an x86 processor. The syntax also keeps this naming convention - DWORD stands for "double word" in Intel syntax and is used for standard-sized registers, which we would call simply a "word".
- Intel assembly language has the ability to address memory as a segment/offset pair. We do not mention this because Linux does not support segmented memory, and is therefore irrelevant to normal Linux programming.

Other differences exist, but they are small in comparison. To show some of the differences, consider the following instruction:

```
movl %eax, 8(%ebx,%edi,4)
```

In Intel syntax, this would be written as:

```
mov [8 + %ebx + 1 * edi], eax
```

The memory reference is a bit easier to read than it's AT&T counterpart because it spells out exactly how the address will be computed. However, but the order of operands in Intel syntax can be confusing.

Where to Go for More Information

Intel has a set of comprehensive guides to their processors. These are available at <http://www.intel.com/design/pentium/manuals/> Note that all of these use the Intel syntax, not the AT&T syntax. The most important ones are their *IA-32 Intel Architecture Software Developer's Manual* in its three volumes::

- Volume 1: System Programming Guide
(<http://developer.intel.com/design/pentium4/manuals/245470.htm>)
- Volume 2: Instruction Set Reference
(<http://developer.intel.com/design/pentium4/manuals/245471.htm>)
- Volume 3: System Programming Guide
(<http://developer.intel.com/design/pentium4/manuals/245472.htm>)

In addition, you can find a lot of information in the manual for the GNU assembler, available online at <http://www.gnu.org/software/binutils/manual/gas-2.9.1/as.html>. Similarly, the manual for the GNU linker is available online at <http://www.gnu.org/software/binutils/manual/ld-2.9.1/ld.html>.

